

The main changes are in the packet headers that the socket buffer provides. For the previous *sk_buff* version (not present in the current version), we could access packet headers via the following structure members (according to `linux.kernel.org`):

```
union h;
union nh;
union mac;
```

In the new version (my machine runs Linux 2.6.31-14-generic), these structure members *h*, *nh* and *mac* are not present, so we have to use a cast. Instead of casting the headers ourselves, we use the following functions.

From *linux/ip.h* for the network header from the buffer:

```
static inline struct iphdr *ip_hdr(const struct sk_buff *skb)
{
    return (struct iphdr *) skb_network_header(skb);
}
```

From *linux/udp.h* for the UDP header from the buffer:

```
static inline struct udphdr *udp_hdr(const struct sk_buff *skb)
{
    return (struct udphdr *) skb_transport_header(skb);
}
```

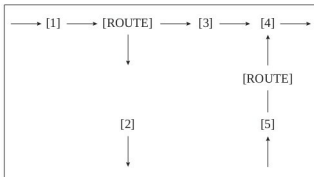
Netfilter

The next step is to understand the Netfilter framework. This is a system of callback functions within the network stack. Callback functions are called for every packet when it reaches certain points while traversing the network stack. Netfilter is a protocol-independent framework, and multiple kernel modules can register with each of the hooks. Netfilter modules can be loaded into the Linux kernel at run time.

With netfilter, we can create a firewall based on stateful or stateless packet filtering. A stateful firewall is one that keeps track of the state of network connections, completely independent of any upper-level state, such as TCP or SCTP state. Part of the reason for this is that when merely forwarding packets (they are not for local delivery), the TCP engine may not necessarily be invoked at all. Even connectionless transmissions such as UDP, IPsec (AH/ESP), GRE (the CISCO tunnelling protocol) and other tunnelling protocols have a connection state (at least pseudo). The firewall is programmed to distinguish packets for different types of connections. Only packets matching a known connection state will be allowed by the firewall, and others will be rejected.

A stateless firewall doesn't check the connection type; it just checks each packet and executes the action(s) it is programmed to carry out.

According to `netfilter.org`, the Netfilter system allows



Schematic diagram 1: Netfilter traversal and hooks

hooks at points 1 to 5 in the schematic above:

- [1] NF_IP_PRE_ROUTING: Here, simple consistency checks are done with regard to the version, length and checksum fields in the IP header. It is useful whenever incoming packets need to be caught before they are processed, like in detecting certain types of Denial of Service attacks.
- [2] NF_IP_LOCAL_IN: All incoming packets addressed to the local computer pass this hook first.
- [3] NF_IP_FORWARD: All incoming packets that are not addressed to the machine, and are to be forwarded (they will leave the computer over a different network interface).
- [4] NF_IP_POST_ROUTING: It represents the last chance to access all outgoing packets (forwarded as well as locally created) before they leave the computer over a network device.
- [5] IF_IP_LOCAL_OUT: All outgoing packets created by the local computer pass this hook.

These are the hook-points defined by Netfilter. For each hook, we can register our own functions defined by us, and they will be called for each packet traversing that point. The callback functions we register must return one of the following values:

- NF_DROP: tells Netfilter to drop the packet, and not to continue its traversal through the system.
- NF_ACCEPT: tells Netfilter to continue traversal as normal.
- NF_STOLEN: tells Netfilter that the filter function has withheld/taken over the packet for further processing, and Netfilter should not continue the packet's traversal.
- NF_QUEUE: tells Netfilter to enqueue the packet to user-space. From this queue, a user-space program can process this packet, and then return it to Netfilter for continued processing.
- NF_REPEAT: calls the hook again. Rather than a continuation of processing at the next packet-filter function, the current filter function is invoked again. This is done if the previous run of the function for the packet failed with some error condition.

Here are some constants for well-known protocols. You